

Ministerul Educației și Cercetării al Republicii Moldova  
Universitatea Tehnică a Moldovei  
FACULTATEA ELECTRONICĂ ȘI TELECOMUNICAȚII  
Departamentul Telecomunicații și Sisteme Electronice

Sarcini practice

## **Tema: Sarcini mostenire polimorfizm**

A elaborat: Creciun Dumitrta  
Grupa: TSRC-251

Chișinău 2026

## Sarcini Mostenire

```
# Clasa de bază
class ContBancar:
    def __init__(self, titular, sold_initial):
        self.titular = titular
        self.sold = sold_initial

    def depunere(self, suma):
        if suma > 0:
            self.sold += suma
            print(f"Depunere de {suma} RON efectuata.")
        else:
            print("Suma invalida pentru depunere.")

    def retragere(self, suma):
        if suma > 0 and suma <= self.sold:
            self.sold -= suma
            print(f"Retragere de {suma} RON efectuata.")
        else:
            print("Fonduri insuficiente sau suma invalida.")

    def afisare_sold(self):
        print(f"Titular: {self.titular} | Sold: {self.sold} RON")

# Clasa derivată - Cont Curent
class ContCurent(ContBancar):
    def retragere(self, suma):
        comision = 5 # comision fix
        if suma > 0 and suma + comision <= self.sold:
            self.sold -= (suma + comision)
            print(f"Retragere: {suma} RON + comision {comision} RON.")
        else:
```

```
26 class ContCurent(ContBancar):
27     def retragere(self, suma):
28         else:
29             print("Fonduri insuficiente pentru retragere + comision.")
30
31
32
33
34
35
36 # Clasa derivată - Cont Economii
37 class ContEconomii(ContBancar):
38     def depunere(self, suma):
39         if suma > 0:
40             bonus = suma * 0.02 # bonus 2%
41             self.sold += suma + bonus
42             print(f"Depunere: {suma} RON + bonus {bonus:.2f} RON.")
43         else:
44             print("Suma invalida pentru depunere.")
45
46
47 # Testare
48 cc = ContCurent("Ion Popescu", 1000)
49 ce = ContEconomii("Maria Ionescu", 2000)
50
51 print("--- Cont Curent ---")
52 cc.afisare_sold()
53 cc.depunere(500)
54 cc.retragere(200)
55 cc.afisare_sold()
56
57 print("\n--- Cont Economii ---")
58 ce.afisare_sold()
59 ce.depunere(500)
60 ce.retragere(300)
61 ce.afisare_sold()
```

```
Retragere de 300 RON efectuata.
Titular: Maria Ionescu | Sold: 2210.0 RON
PS C:\Users\creci>
```

## Sarcini Polimorfizm

C: > Users > creci > Python > # Clasa de bază.py > ...

```
1  # Clasa de bază
2  class Plata:
3      def executa_plata(self, suma):
4          print("Se executa o plata generica.")
5
6
7  # Clasa derivată - Plata Card
8  class PlataCard(Plata):
9      def executa_plata(self, suma):
10         print(f"Plata de {suma} RON efectuata cu cardul.")
11
12
13  # Clasa derivată - Plata Cash
14  class PlataCash(Plata):
15      def executa_plata(self, suma):
16         print(f"Plata de {suma} RON efectuata cash.")
17
18
19  # Testare polimorfism
20  plati = [
21      PlataCard(),
22      PlataCash(),
23      Plata() # opțional, clasa de bază
24  ]
25
26  for plata in plati:
27      plata.executa_plata(100)
```

```
Plata de 100 RON efectuata cash.
Se executa o plata generica.
PS C:\Users\creci>
```